

Progress Report

2026-04-09

Members

Kushal Agrawal

Eeshana Hari

Current Progress

The team has brought the dual-FPGA trading engine from specification to exercised RTL. On the *verification* side, we built a Python golden model and exported structured test vectors (JSON) so that RTL simulation can be checked against known-good arithmetic for exchange behavior, feature computation, and other data-plane paths. That gives us a single source of truth for “what should happen” when the same seeds and parameters are applied in hardware and software.

On the *implementation* side, all major PL blocks are in place for both boards: Board A (market simulation, exchange, arbitration, and control), Board B (quote handling through strategy, risk, orders, positions, and telemetry registers), the shared PMOD link layer, and shared primitives (LFSR, FIFOs, debounce, etc.). PS-facing logic is integrated through AXI-Lite register files (Framework laid out, yet to know if functionally correct). We have run extensive **unit tests** per module and **integration tests** that stitch full `board_a.top` and `board_b.top` hierarchies, including a full-system testbench and comparison against golden vectors where applicable.

On the hardware side, we have moved past pure RTL-only validation: bring-up has started on AUP-ZU3. So far we have validated the tool flow with an LED blinker, and **successfully generated bitstreams** from Vivado for our design. The next milestones are to stabilize the Board A block design and AXI path to our packaged IP, repeat the flow for Board B, then connect the boards over PMOD and run the PYNQ scripts (`config_symbols.py`, `telemetry_server.py`) end-to-end.

Testing

Software (golden model and vectors)

The golden model lives under `golden_model/` with per-block vector files in `golden_model/vectors/` (e.g., exchange, features, LFSR). Those files capture expected outputs for chosen seeds and configuration so that simulation and (eventually) se-

lective hardware checks can be compared without hand-calculating Q-format math. Generating vectors once and reusing them across regressions reduced disagreement between teammates and made integration failures easier to localize (RTL vs. testbench vs. vector).

Simulation (unit, integration, system)

Every major RTL subtree has a dedicated SystemVerilog testbench under `simulation_tb/`: shared blocks (LFSR, debounce, synchronous FIFO), link TX/RX and loopback, Board A pipeline pieces (market sim, exchange, arbiter), and Board B (demux, quote book, features, strategy, risk, order manager, position tracker, histogram, etc.). We also run **cross-cutting tests**: for example, link loopback proves the serializer and receiver together; `tb_system.top` exercises the full split design in simulation. Most tests are self-checking (PASS/FAIL and error counts). Remaining work on simulation is mostly **final integration hardening**: occasional tool or timing-report noise during long runs, and closing any last corner cases before we freeze RTL for the demo.

Hardware (TODO)

Hardware validation is the main focus for this reporting period and the next. Current blockers include confirming that the **Vivado block design for Board A** matches our intent (clock/reset/AXI connectivity) and resolving **AXI read/write** behavior through the interconnect into our custom IP—we need deterministic MMIO from Linux before standalone Board A tests are meaningful. Synthesis and bitstream generation are already in the loop; the emphasis shifts to **deploying .bit/.hwh** pairs on each board and stepping through the lab’s bring-up phases.

Concrete next steps (aligned with our internal checklist and course milestones):

- A1.** Finish debugging AXI and wrapper wiring for Board A; timing is tentatively meeting goals at 100 MHz but we continue to verify on hardware.
- A2.** Reload overlay on Board A and run `config_symbols.py` in a standalone configuration (status registers, quote activity, no dependency on Board B yet).
- A3.** Use slide switches to change regime or related

Progress Report

2026-04-09

controls and confirm visible changes in status or quote behavior per the spec.

- B1.** Complete Board B Vivado build (block design + same PS shell pattern as Board A).
- B2.** Repeat LED and AXI smoke tests on Board B before trusting larger tests.
- B3.** Deploy Board B `.bit` and `.hwh` to the SD card overlay path expected by PYNQ.
- S1.** Connect Board A and Board B via PMOD; verify `link_up`, FSM states, and that `telemetry_server.py` streams sensible JSON.
- S2.** Compare quote throughput: Board B `quotes_rcvd` vs Board A `quotes_sent` (allowing small skew while both sides run).
- S3.** Raise `trading_enable` (switch) and confirm orders and fills propagate as designed.
- S4.** Attach the laptop `dashboard.py` to Board B's UART and verify live plots at the target baud rate.

Individual Contributions

Kushal Agrawal

Led the **golden model** effort: structure of the Python reference, vector generation, and alignment with `hft_pkg` conventions (fixed-point formats, message layouts). Owns much of the **Board A integration testing** story (top-level testbench usage, regression triage) and **Board B** RTL in several subsystems plus shared peripherals `debounce.sv` and `sync_fifo.sv`, each with unit tests. Has been driving **Board B integration** simulation and will continue supporting **Board B hardware bring-up** (overlay load, MMIO, telemetry path) in parallel with the checklist above.

Outstanding

Close out any remaining full-chip integration test gaps; complete Board B on-hardware steps (B1–B3, then S1–S4 as the link comes up).

Eeshana Hari

Primary ownership of **Board A RTL** modules and their **unit testbenches**, from control and datapath glue through exchange-side behavior. Implemented and tested the **link layer** blocks and the loopback testing. Implemented the shared

`lfsr32.sv` primitive used by the noise and simulation flows, with matching TB and golden vectors.

Outstanding

Lead **Board A hardware bring-up**: iterating through Vivado IP packaging, AXI interconnect hookup, block-design layout, and on-board validation. Will coordinate with Kushal when the PMOD link joins both boards for system-level tests.